

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 6 September 2013 - Part I: Theory

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

SURNAME:

ID:

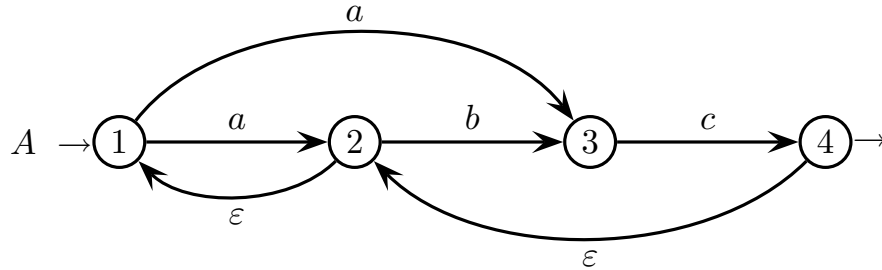
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following finite automaton A (which is not deterministic) over the alphabet $\{a, b, c\}$:



Answer the following questions:

- (a) Draw a finite automaton B equivalent to A , but without spontaneous transitions.
 - (b) Draw a deterministic finite automaton C equivalent to A .
 - (c) Draw a deterministic finite automaton D that recognizes the complement language $\overline{L(A)}$, verify that it is minimal and, if necessary, minimize it.
 - (d) (optional) Write a non-ambiguous regular expression R that generates the language $L(A)$.
-

2 Free Grammars and Pushdown Automata 20%

1. The language L of the polish prefix expressions with the two operators $\oplus$ and $\ominus$, which are both binary (that is with two operands), and with the operand a , is defined by the following grammar G (axiom S):

$$G: S \rightarrow \oplus S S \mid \ominus S S \mid a$$

Consider also the two regular expressions R_1 and R_2 below:

$$R_1 = \{ \oplus, a \}^* \ominus \{ \oplus, a \}^*$$

$$R_2 = (\oplus a^* \ominus a^*)^*$$

Answer the following questions:

- (a) Explain informally (that is by words) what the condition expressed by the regular expression R_1 is. Then write a non-extended (*BNF*) and non-ambiguous grammar G_1 for the intersection language $L \cap L(R_1)$, and draw the syntax tree of the sample string $\oplus \oplus a \ominus a \oplus a a a$.
 - (b) (optional) Write a non-extended (*BNF*) grammar G_2 for the intersection language $L \cap L(R_2)$.
-

2. A hypothetical language for vectored graphic description (e.g., Picture in MS Word) comprises the following basic shapes: triangle (*triang*) and polygon (*polygon*).

All the basic shapes must begin with an attribute *points* for listing, between commas and separated by a slash, pairs of natural numbers (with their left and right members separated by a comma) that represent the coordinates of the vertices of the shape (3 coord. pairs for a triangle and > 3 coord. pairs for a polygon).

Furthermore every basic shape may have an attribute *style* for listing, separated by a semicolon, the following aspects (each in the form *aspect:value*):

fill filling color; the possible color values are *black*, *red*, *green* and *none*; suppose that to each color there corresponds a suitable terminal symbol

stroke color of the contour line (same color values as above)

width thickness of the contour line, the value of which is a natural number; suppose that the terminal symbol *n* represents any natural number

Each aspect is optional, but the aspect list - if the attribute *style* is present - may not be empty; the listing order of the aspects is fixed as above.

Here are a few basic sample shapes:

```
triangle points = "1,2 / 3,4 / 5,6" end triangle
```

```
triangle points = "1,3 / 5,7 / 4,6"  
  style = fill:red; stroke:red; width:3  
end triangle
```

```
polygon points = "1,1 / 2,3 / 2,5 / 6,5"  
  style = fill:green; width:2  
end polygon
```

Among the possible shapes there is also the two-level polygon, the vertices of which are occupied by basic shapes. The description of a two-level polygon must contain a number ≥ 3 of basic shapes, though described only as for their geometry - that is without attribute *style* - and may contain only one attribute *style*, which is valid for the whole two-level shape. Here is a sample two-level shape:

```
poly2level  
  
  triangle points = "1,3 / 5,7 / 4,6" end triangle  
  
  polygon points = "2,3 / 5,9 / 4,7" end polygon  
  
  polygon points = "1,3 / 5,7 / 4,6 / 15,16" end polygon  
  ...  
  style = fill:green; stroke:red; width:2  
  
end poly2level
```

Last, among the possible shapes there is the group, which contains a number ≥ 2 of shapes of any type: basic, two-level polygon and group. Here is a sample group:

```
group
  polygon
    ...
  end polygon

  group
    ...
  end group

  poly2level
    ...
  end poly2level

end group
```

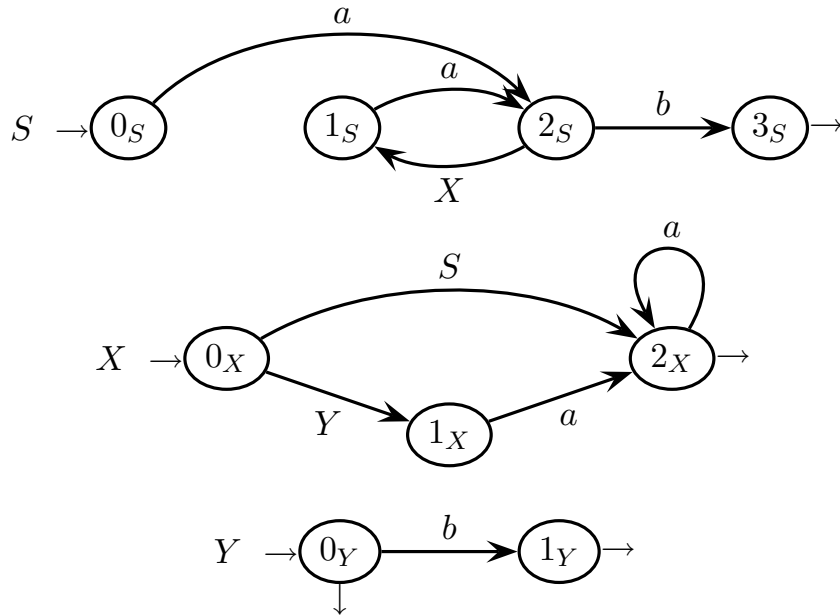
Concluding, a string of this description language consists of a non-empty sequence of shapes of any type (basic, two-level polygon and group).

Answer the following questions:

- (a) Write an extended (*EBNF*) and non-ambiguous grammar G_1 that models the description language hinted above, and suppose that the only basic shape is the triangle and that there are not any two-level polygons.
 - (b) (optional) Write an extended (*EBNF*) and non-ambiguous grammar G_2 that comprises also the basic polygon and the two-level polygon.
-

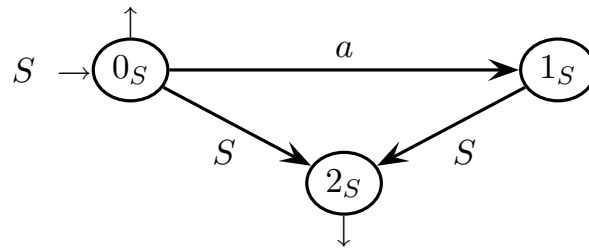
3 Syntax Analysis and Parsing 20%

1. Consider the following extended (*EBNF*) grammar G , which is represented as a network of recursive machines over the alphabet $\{ a, b, S, X, Y \}$:



Answer the following questions:

- (a) Draw the whole *ELR*(1) pilot graph of grammar G and list all the *ELR* conflicts it contains.
- (b) Examine the string $abaaab \in L(G)$ by means of the Earley algorithm applied to grammar G , and draw the syntax tree. Use the frame on the next pages.
- (c) (optional) Consider the following extended (*EBNF*) grammar G' , which is represented as one recursive machine over the alphabet $\{ a, S \}$:



Grammar G' is ambiguous: write an ambiguous string generated by G' with two (or more) derivations. Grammar G' has a conflict of type *ELR*(1): explain what conflict type it is.

CAUTION: read the warning on the cover page.

Earley vector for string $a_1 b_2 a_3 a_4 b_5$

[illegible]

syntax tree of string $abab$

4 Translation and Semantic Analysis 20%

1. Consider the following translation $\tau: a^* \rightarrow q^* r^*$:

$$\tau(a^n) = q^{(n \div 3)} r^{(n \bmod 3)} \quad n \geq 0$$

where the operators “ $\div$ ” and “ $\bmod$ ” compute the quotient and the remainder of the integer division, respectively. For instance:

$$\begin{aligned} \tau(\varepsilon) &= \varepsilon \\ \tau(a) &= r \\ \tau(aa) &= \tau(a^2) = r^2 = r r \\ \tau(aaa) &= \tau(a^3) = q \\ \tau(aaaa) &= \tau(a^4) = q r \\ \tau(aaaaa) &= \tau(a^5) = q r^2 = q r r \\ \tau(aaaaaa) &= \tau(a^6) = q^2 = q q \end{aligned}$$

Answer the following questions:

- (a) Draw a deterministic sequential transducer T that computes the translation τ described above.
- (b) Draw a finite automaton A with two input tapes that recognizes the string pairs (x, y) , where it holds:

$$x = a^n \quad y = (\tau(x))^R = r^{(n \bmod 3)} q^{(n \div 3)} \quad n \geq 0$$

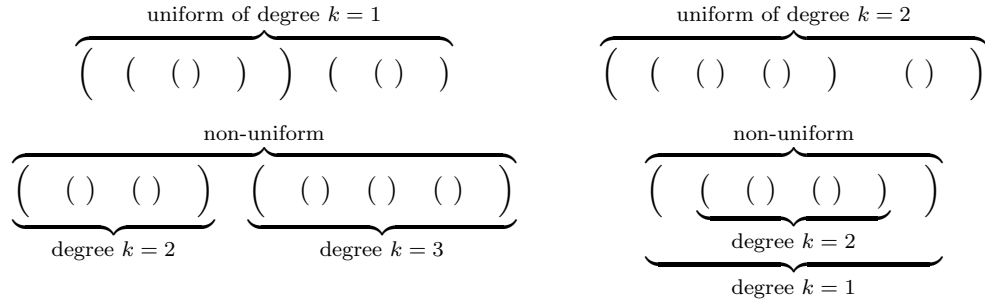
that is, string y is the reflected image of the translation τ of x . A sample pair is $(aaaaa, rrrq) = (a^5, r^2q)$.

- (c) (optional) Write a translation grammar (or a scheme) G_τ equivalent to the transducer T found at point (a).

2. Consider the nested parenthesis structures generated by the following syntax:

$$A \rightarrow A B \mid B \qquad B \rightarrow '(' A ')' \mid '(' ' ')'$$

A parenthesis structure is *uniform of degree $k \geq 1$* if every non-empty pair of matching parentheses immediately and exactly contains k parenthesis pairs. The samples below exhibit two uniform structures of degree 1 and 2, and two non-uniform ones.



Add the following axiomatic rule (axiom S) to the syntax above:

$$S \rightarrow A k$$

where the terminal symbol k specifies a degree ≥ 1 . This terminal can be identified to an attribute with the same name k .

Answer the following questions:

- (a) Write an attribute grammar to verify that a parenthesis structure is uniform of degree k . Use the following attributes:

<i>type</i>	<i>name</i>	<i>association</i>	<i>domain</i>	<i>meaning</i>
right	k	k	integer	requested degree to be verified on the structure
right	d	$A B$	integer	degree to be propagated and verified on the structure
left	u	$A B S$	boolean	$u = \text{true}$ if the structure is uniform of degree k , else $u = \text{false}$

It is permitted to add new attributes, if it is useful or necessary (also for question (b)). Write the attributes and the semantic rules in the frames on the next pages.

- (b) (optional) One wishes one computed a new attribute called *error*, which gives the position of the leftmost closed bracket where the uniformity condition appears to be violated. Attribute *error*, synthesized, is visible at the tree root S and in the absence of errors it holds $error = 0$. So in the example below, by positing $k = 2$ such a bracket has position $error = 8$:

$$\left(\begin{array}{ccccccc} & (& (& (&) & (&) &) &) \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{array} \right)$$

because 2-7 has degree 2 = k , but 1-8 has degree 1 $\neq k$ (3-4 and 5-6 are empty and do not matter). Write the semantic rules, in the frame on the next pages.

List of the predefined attributes and of the possible attributes to add or complete for questions (a) and (b) (the number of rows is not significant):

<i>type</i>	<i>name</i>	<i>association</i>	<i>domain</i>	<i>meaning</i>
right	k	k	integer	requested degree to be verified on the structure
right	d	$A \ B$	integer	degree to be propagated and verified on the structure
left	u	$A \ B \ S$	boolean	$u = true$ if the structure is uniform of degree k , else $u = false$
left	$error$	$A \ B \ S$	integer	position (≥ 1) of the leftmost closed bracket where the uniformity condition appears to be violated; in the absence of errors it holds $error = 0$

#	<i>syntax</i>	<i>semantic</i> - question (a) - to be completed
---	---------------	--

$d_1 := k_2$
 $u_0 := u_1$

1: $S_0 \rightarrow A_1 k_2$

2: $A_0 \rightarrow A_1 B_2$

3: $A_0 \rightarrow B_1$

4: $B_0 \rightarrow (A_1)$

$u_0 := true$

5: $B_0 \rightarrow ()$

#	<i>syntax</i>	<i>semantic</i> - question (b) - to be completed
		$d_1 := k_2$ $u_0 := u_1$ $error_0 := error_1$
1:	$S_0 \rightarrow A_1 k_2$	
2:	$A_0 \rightarrow A_1 B_2$	
3:	$A_0 \rightarrow B_1$	
4:	$B_0 \rightarrow (A_1)$	
		$u_0 := true$ $error_0 := 0$
5:	$B_0 \rightarrow ()$	

